R.No: 34 Reg. No: 192425176 Name: SOMISETTY VENKATA SAI NAVEEN KUMAR

Answer all 5 questions (5 x 10 = 50 marks) on A4 sheets. Programs must be in R (ggplot2 and the packages named). Show every calculation; answers without working or without reference to the given figures earn no marks. Your data differ from every other student's.

Q1. Hexagonal binning of an overplotted scatter [10 marks] *(Hexbin Plots)*

1500 points of output (units per shift) versus defect rate (%) are drawn on a 60 x 50 unit panel. Hexagons of side 3 units are proposed. A coarse 3 x 4 rectangular count of the same points is below (rows = y bands, columns = x bands):

	x1	x2	x3	x4
y1	48	132	129	111
y2	150	120	105	126
y3	141	99	108	81

(a) Compute the area of one regular hexagon of side s (3*sqrt(3)/2 * s^2) and the approximate number of hexagons needed to cover the panel. [3]

(b) Compute the average number of points per hexagon, and the number per unit area. [2]

(c) From the table, compute the total counted, the share of the busiest cell and its ratio to the average cell count (average = total/12). [3]

(d) Write ggplot2 code with geom_hex(bins = 25) and a viridis fill with log10 transform; say why hexagons beat squares. [2]

Q2. Scatterplots, correlation and outliers [10 marks] *(Scotterplots)*

A manufacturing plant studies how output (units per shift) (y) depends on downtime (minutes per shift) (x) across 8 production lines. Data:

production lines	x	y
Line H	2	31
Line A	3	42
Line F	5	41
Line G	6	46
Line D	10	33
Line C	12	69
Line B	13	68
Line E	15	75

(a) Compute the means of x and y, then the least-squares slope b = Sxy/Sxx and intercept a. [3]

(b) Compute Pearson's r = Sxy / sqrt(Sxx * Syy) and say what r^2 means here. [3]

(c) Find the largest absolute residual. Name the point, give its residual and the correlation r when that point is removed; comment on its influence. [2]

(d) Write ggplot2 code for the scatterplot with geom_smooth(method = 'lm'), the outlier labelled with ggrepel and a loess curve in a second colour. [2]

Q3. Moving-window and exponential smoothing [10 marks] *(Smoothing Techniques)*

A daily series of output (units per shift) at a manufacturing plant contains one short real spike:

Day	1	2	3	4	5	6	7	8	9
Value	533	509	713	501	539	537	533	499	532

(a) Compute the centred 3-day moving average for days 2 to 8. [3]

(b) Compute exponential smoothing s_t = a*y_t + (1 - a)*s_(t-1) with a = 0.5 and s_1 = y_1 for all 9 days. [3]

(c) The spike is on day 3. Compare the spike height (value minus the mean of the other days) in the raw series and in the 3-day average; what share survives, and what does a 7-day window do (at most 1/7)? [2]

(d) Write ggplot2 code overlaying raw points, a loess curve with span = 0.3 and the exponential-smoothing line. [2]

Q4. Simple exponential smoothing and forecast intervals [10 marks] *(Forecasting Trends)*

Monthly output (units per shift) at a manufacturing plant for six months:

Month	1	2	3	4	5	6
Value	532	538	548	561	572	580

(a) Apply simple exponential smoothing with a = 0.3, starting with forecast F_2 = y_1; compute F_3..F_7 using F_(t+1) = a*y_t + (1-a)*F_t. [3]

(b) Compute the one-step errors for months 2 to 6 and the MAE; compare with the naive method (forecast = last value), whose MAE is the mean absolute first difference. [3]

(c) The residual standard deviation is about 2.70. For a naive forecast an approximate 95% interval is +/- 1.96 * s * sqrt(h). Compute its half-width at h = 1 and h = 9 and explain why the band widens. [2]

(d) Write R code with forecast::ses or HoltWinters and autoplot with the 95% band. [2]

Q5. Choropleth maps: counts versus rates and class breaks [10 marks] *(Choropleth Maps)*

A manufacturing plant counted units produced in six regions:

Region	Population	Count
Border	240,000	2732
West	6e+05	3408
Coastal	9e+05	3775
North	120,000	995
Central	1,200,000	12174

Plateau	480,000	2130

(a) Compute the rate per 10,000 people in each region. [3]
(b) Rank the regions by raw count and by rate and name the region whose rank changes most; explain the risk of mapping raw counts. [3]
(c) Compute equal-interval breaks for three classes (Low, Medium, High) from the minimum and maximum rate and assign each region to a class. [3]
(d) Write ggplot2 code: geom_sf(aes(fill = rate)) with scale_fill_viridis_b or scale_fill_fermenter. [1]

Q1. Hexagonal binning of an overplotted scatter

(a) Area of one hexagon = (3 x sqrt(3) / 2) x s^2 = 2.598 x 9 = 23.38 square units. Panel area = 60 x 50 = 3000 square units, so the number of hexagons needed is 3000 / 23.38 = 128.3, which is about 129 hexagons.

(b) Average points per hexagon = 1500 / 128.3 = 11.7 (11.6 if 129 hexagons are used). Points per unit area = 1500 / 3000 = 0.5.

(c) Row totals: y1 = 420, y2 = 501, y3 = 429, so the total counted = 1350. The busiest cell is y2, x1 with 150 points, a share of 150 / 1350 = 11.1%. The average cell count = 1350 / 12 = 112.5, so the busiest cell is 150 / 112.5 = 1.33 times the average.

(d) Code:

```
library(ggplot2)
ggplot(df, aes(x, y)) +
  geom_hex(bins = 25) +
  scale_fill_viridis_c(trans = "log10", name = "Count (log10)") +
  labs(x = "Output (units per shift)", y = "Defect rate (%)")
```

Hexagons beat squares because all six neighbours of a hexagon are the same distance from its centre, so there is no bias toward horizontal or vertical patterns, the shape is closer to a circle (less edge distortion), and density changes look smoother.

Q2. Scatterplots, correlation and outliers

(a) Mean of x = 66 / 8 = 8.25 and mean of y = 405 / 8 = 50.625. Sxx = 167.5 and Sxy = 494.75, so the slope b = 494.75 / 167.5 = 2.954 and the intercept a = 50.625 - 2.954 x 8.25 = 26.26. Fitted line: y = 26.26 + 2.954x.

(b) Syy = 2117.875, so r = 494.75 / sqrt(167.5 x 2117.875) = 494.75 / 595.6 = 0.831. r^2 = 0.690, which means about 69% of the variation in output across the lines is explained by its linear relationship with downtime.

(c) The largest absolute residual is Line D (x = 10, y = 33): fitted value 55.79, residual = 33 - 55.79 = -22.79. With Line D removed, r rises from 0.831 to 0.986, so this one point is highly influential and weakens the correlation considerably.

(d) Code:

```
library(ggplot2); library(ggrepel)
df$resid <- resid(lm(y ~ x, df))
out <- df[which.max(abs(df$resid)), ]
ggplot(df, aes(x, y)) + geom_point() +
  geom_smooth(method = "lm", se = FALSE, colour = "blue") +
  geom_smooth(method = "loess", se = FALSE, colour = "red", linetype = "dashed") +
  geom_text_repel(data = out, aes(label = `production lines`)) +
  labs(x = "Downtime (minutes per shift)", y = "Output (units per shift)")
```

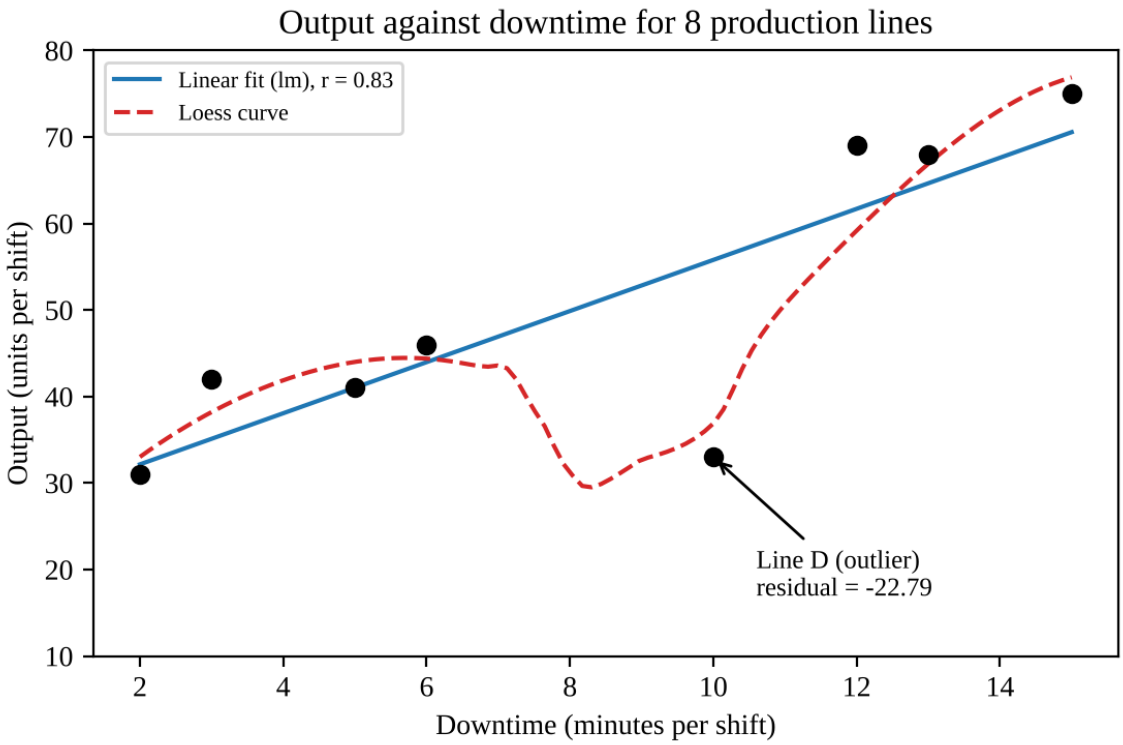


Figure 1. Scatterplot with linear fit, loess curve and the labelled outlier (Line D).

Q3. Moving-window and exponential smoothing

(a) Centred 3-day moving average: day 2 = (533 + 509 + 713) / 3 = 585.00; day 3 = 574.33; day 4 = 584.33; day 5 = 525.67; day 6 = 536.33; day 7 = 523.00; day 8 = 521.33.

(b) Exponential smoothing with a = 0.5 (s_1 = 533): s_2 = 521.00, s_3 = 617.00, s_4 = 559.00, s_5 = 549.00, s_6 = 543.00, s_7 = 538.00, s_8 = 518.50, s_9 = 525.25.

(c) Raw spike height = 713 - mean of the other 8 days (522.88) = 190.1. In the 3-day average the day-3 value is 574.33 and the mean of the other averaged days is 545.94, so the spike height is 28.4, which is about 15% of the raw spike. The average spreads the spike over days 2 to 4, so a 3-day window keeps at most one third of it. A 7-day window keeps at most 1/7 (about 14%), so it would flatten the real spike even more.

(d) Code:

```
library(ggplot2)
df$es <- Reduce(function(prev, y) 0.5 * y + 0.5 * prev, df$value[-1], accumulate = TRUE,
init = df$value[1])
ggplot(df, aes(day, value)) + geom_point() +
  geom_smooth(method = "loess", span = 0.3, se = FALSE, colour = "darkgreen") +
  geom_line(aes(y = es), colour = "red")
```

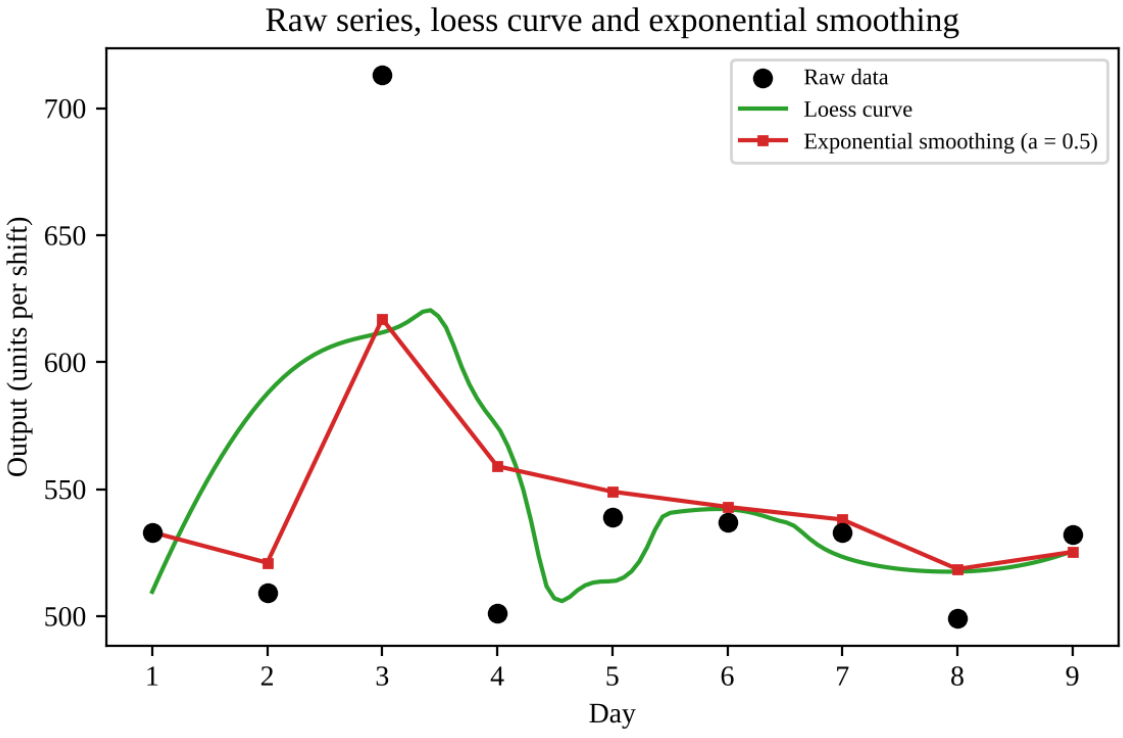


Figure 2. Raw series, loess curve and exponential-smoothing line (loess shown with a wider span of 0.6, because a span of 0.3 is too small to fit only 9 points).

Q4. Simple exponential smoothing and forecast intervals

(a) With a = 0.3 and F_2 = 532: F_3 = 0.3(538) + 0.7(532) = 533.80; F_4 = 538.06; F_5 = 544.94; F_6 = 553.06; F_7 = 561.14.

(b) One-step errors for months 2 to 6: 538 - 532 = 6.00; 548 - 533.80 = 14.20; 561 - 538.06 = 22.94; 572 - 544.94 = 27.06; 580 - 553.06 = 26.94. MAE = 97.14 / 5 = 19.43. The naive method has absolute errors 6, 10, 13, 11, 8, so its MAE = 48 / 5 = 9.6. The naive method is better here because the series trends upward and simple exponential smoothing lags behind it.

(c) Half-width = 1.96 x 2.70 x sqrt(h): at h = 1 it is 5.29 and at h = 9 it is 1.96 x 2.70 x 3 = 15.88. The band widens because forecast errors accumulate: the variance grows in proportion to h, so the standard deviation grows with sqrt(h) and uncertainty is about 3 times larger nine steps ahead.

(d) Code:

```
library(forecast)
y <- ts(c(532, 538, 548, 561, 572, 580), start = 1)
fit <- ses(y, alpha = 0.3, initial = "simple", h = 9, level = 95)
autoplot(fit) + labs(x = "Month", y = "Output (units per shift)")
```

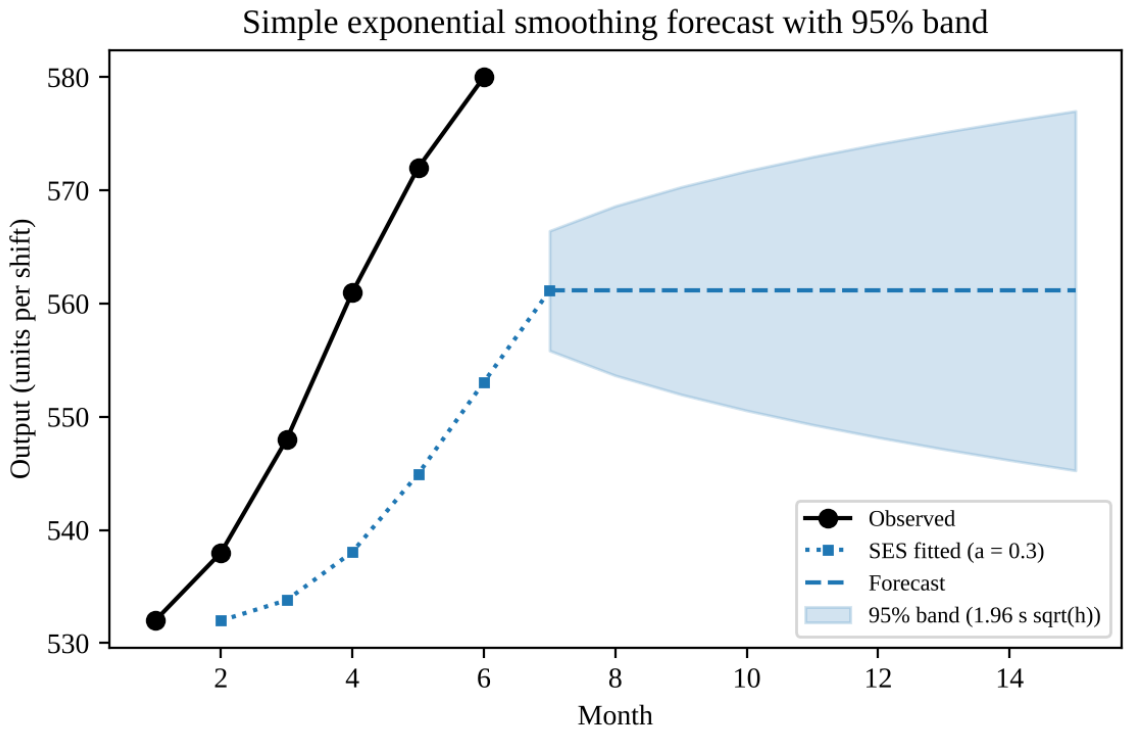


Figure 3. Simple exponential smoothing forecast with the approximate 95% band.

Q5. Choropleth maps: counts versus rates and class breaks

(a) Rate per 10,000 = count / population x 10,000:

Region	Population	Count	Rate per 10,000
Border	240,000	2732	113.83
West	600,000	3408	56.80
Coastal	900,000	3775	41.94
North	120,000	995	82.92
Central	1,200,000	12174	101.45
Plateau	480,000	2130	44.38

(b) Rank by raw count: Central 1, Coastal 2, West 3, Border 4, Plateau 5, North 6. Rank by rate: Border 1, Central 2, North 3, West 4, Plateau 5, Coastal 6. Coastal changes most (rank 2 by count to rank 6 by rate, a drop of 4 places); Border and North each move up 3 places. Mapping raw counts is risky because counts mostly reflect population size: Central (12174) and Coastal (3775) look like hot spots only because they are large, while the small Border and North regions with high rates would look unimportant.

(c) Minimum rate = 41.94 (Coastal) and maximum = 113.83 (Border), so the class width = (113.83 - 41.94) / 3 = 23.96. Breaks: 41.94, 65.91, 89.87, 113.83. Low (41.94 to 65.91): Coastal, Plateau, West. Medium (65.91 to 89.87): North. High (89.87 to 113.83): Central, Border.

(d) Code:

```
library(ggplot2); library(sf)
ggplot(regions_sf) +
  geom_sf(aes(fill = rate), colour = "white") +
  scale_fill_viridis_b(name = "Rate per 10,000", n.breaks = 4) +
  theme_void()
# alternative: + scale_fill_fermenter(palette = "YlGnBu", direction = 1, n.breaks = 4)
```